```
nice -n 10 grind          # 降低普通进程的时间片
chrt -f -p 60 6           # 修改 PID 6（某个 kworker）的策略
```

8.4 用户线程

8.4.1 设计

原来的模型是“一个 struct proc 对应一个独立的地址空间”。加入线程时，项目没有引入新的“线程控制块”，而是让每个线程仍然是一个完整的 struct proc——有自己的内核栈、trapframe、调度属性、TID 和 CPU 亲和性——只是同一线程组的成员共享同一个引用计数的 struct vmspace（图 8.1）。

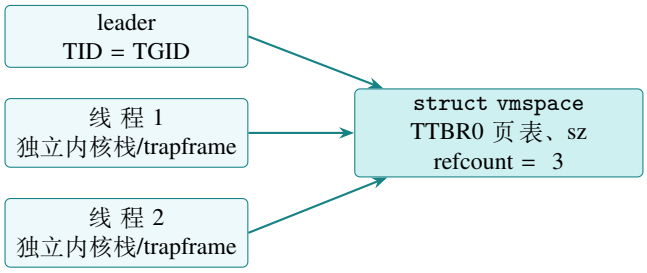


图 8.1: 线程组共享 vmspace

调度器切换到任何一个线程时，switchuvm() 都写入 p->vm->pagetable，所以同组线程看到相同的代码、全局变量、堆和彼此的栈；而它们的内核栈和 trapframe 各自独立，可以同时在多个 Cortex-A53 核上运行。

8.4.2 系统调用

表 8.2: 线程相关系统调用

调用	作用
clone(entry, arg, stack_top)	创建共享地址空间的新调度实体：x0=arg，SP=stack_top，ELR=entry
texit(status)	只退出调用线程；leader 调用时等同于整个进程退出
tjoin(tid, &status)	等待并回收同组中的指定线程（0 表示任意）
gettid() / getpid()	返回线程 ID / 线程组 ID

用户库在其上提供更安全的包装，它负责分配 16 字节对齐的 16 KiB 用户栈，并保证线程函数返回后自动调用 texit()：

```
static void thread_boot(void *opaque) {
  struct thread_start start = *(struct thread_start *)opaque;
  free(opaque);                       // 启动记录用完即释放
  start.fn(start.arg);
  texit(0);                           // 线程函数返回后自动退出
}

int thread_create(thread_t *thread, void (*fn)(void *), void *arg) {
  thread->stack = malloc(THREAD_STACK_SIZE + 16);
  struct thread_start *start = malloc(sizeof(*start));
  start->fn = fn;  start->arg = arg;
  uint64 top = ((uint64)thread->stack + THREAD_STACK_SIZE + 16) & ~15ULL;
  thread->tid = clone(thread_boot, start, (void *)top);
```